

LeetCode 84: Largest Rectangle in Histogram

Complete Problem Analysis, Conceptual Framework & Implementation

Authored by: Arpit Raj • Institution: LNMIIT Jaipur • Date: July 9, 2026

Frequently Asked In: Meesho, Zomato, Sprinklr, Moq Software

Problem Overview & Conceptualization

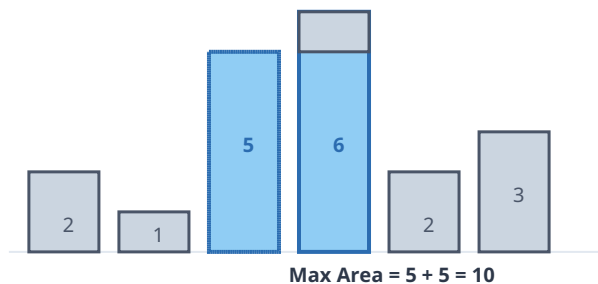
Given an array of integers `heights` representing the histogram's bar height where the width of each bar is 1, find the area of the largest rectangle in the histogram.

Example Walkthrough:

`heights = [2, 1, 5, 6, 2, 3]`

The largest rectangle is formed by the bars of height 5 and 6, giving a total width of 2.

Area = $5 \times 2 = 10$



To compute the maximum area for a given bar at index i assuming `heights[i]` is the limiting (minimum) height, we must determine how far the rectangle can expand to the left and right. The boundaries are determined by the first element strictly smaller than `heights[i]` on both sides:

- **Previous Smaller Element (l):** The first index to the left where `heights[l] < heights[i]`.
- **Next Smaller Element (r):** The first index to the right where `heights[r] < heights[i]`.

The width of the rectangle is given by: **$width = r - l - 1$**

Approach 1: Brute Force (Expansion Method)

For every bar, expand left and right until a bar with a smaller height is encountered. Compute the area for each bar and record the maximum.

Time Complexity: $O(n^2)$ • **Space Complexity:** $O(1)$

```
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {

        int n=heights.size();

        int ans=0;

        for(int i=0;i<n;i++){

            // Assume heights[i] is the minimum height

            int l=i;
            int r=i;

            // Expand towards left
            while(l-1>=0 && heights[l-1]>=heights[i]){
                l--;
            }

            // Expand towards right
            while(r+1<n && heights[r+1]>=heights[i]){
                r++;
            }

            int width=r-l+1;

            int area=width*heights[i];

            ans=max(ans,area);
        }

        return ans;
    }
};
```

Approach 2: Monotonic Stack (Precomputing Previous & Next Smaller)

Using a monotonic stack, we precompute two vectors: prev and next to store the indices of the previous smaller element and next smaller element respectively. If no smaller element exists on the left, we default to -1. If none exists on the right, we default to n.

Time Complexity: $O(n)$ • **Space Complexity:** $O(n)$

```

class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        int n=heights.size();
        vector<int> prev(n);
        vector<int> next(n);
        stack<int> st;

        for(int i=0;i<n;i++){
            while(!st.empty() && heights[st.top()]>=heights[i]){
                st.pop();
            }

            if(st.empty()){
                prev[i]=-1;
            }

            else{
                prev[i]=st.top();
            }

            st.push(i);
        }

        while(!st.empty()){ //reuse krenge
            st.pop();
        }

        for(int i=n-1;i>=0;i--){
            while(!st.empty() && heights[st.top()]>=heights[i]){
                st.pop();
            }

            if(st.empty()){
                next[i]=n;
            }

            else{
                next[i]=st.top();
            }

            st.push(i);
        }

        int ans=0;

        for(int i=0;i<n;i++){
            int w=next[i]-prev[i]-1;
            int h=heights[i];
            ans=max(ans,w*h);
        }

        return ans;
    }
};

```

```
}  
};
```

| Approach 3: Monotonic Stack (Single Pass Optimized)

Instead of doing multiple passes to find the left and right boundaries separately, we can compute the area in a single linear pass. When an element is popped from the stack, it acts as the minimum height bar. The current index i serves as its right boundary, and the element currently at the top of the stack serves as its left boundary.

Time Complexity: $O(n)$ • **Space Complexity:** $O(n)$

```

class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {

        int n=heights.size();

        stack<int> st;

        int ans=0;

        for(int i=0;i<=n;i++){

            while(!st.empty() && (i==n || heights[st.top()]>=heights[i])){

                int h=heights[st.top()];
                st.pop();

                int right=i;

                int left;

                if(st.empty()){
                    left=-1;
                }
                else{
                    left=st.top();
                }

                int width=right-left-1;

                ans=max(ans,h*width);
            }

            st.push(i);
        }

        return ans;
    }
};

```